
Guiding Posterior Exploration with Optimizer-Derived Geometry

Moritz Schlager*
TUM, MCML

Emanuel Sommer*
LMU Munich, MCML

Thomas Möllenhoff
RIKEN AIP

David Rügamer
LMU Munich, MCML

Abstract

Sampling-based methods offer a principled approach to uncertainty quantification in Bayesian neural networks. Their practical use, however, is often challenged by the computational cost of exploring high-dimensional and multimodal posterior distributions. To overcome these difficulties, Bayesian Deep Ensembles, i.e., warmstarting the sampling from several optimized solutions, have proven to be an effective strategy. In this paper, we demonstrate that curvature estimates computed during the warmstart as a byproduct in adaptive optimizers such as AdamW can inform the sampling phase at negligible additional cost. Specifically, our proposed preconditioned sampling strategy based on optimizer-derived geometries can substantially reduce or even eliminate the need for a lengthy sampling burn-in phase and leads to greater numerical stability. This approach consistently maintains or improves predictive performance and uncertainty quantification without any additional computational costs. We confirm the consistency of our findings across various datasets and network architectures.

1 INTRODUCTION

Bayesian deep learning strives to build a principled framework for uncertainty quantification, a critical requirement for deploying neural networks in high-stakes domains. Sampling-based methods such as Markov chain Monte Carlo (MCMC) offer a powerful means of approximating the posterior distribution over the model’s parameters. However, their practical application is often hindered by the immense computational

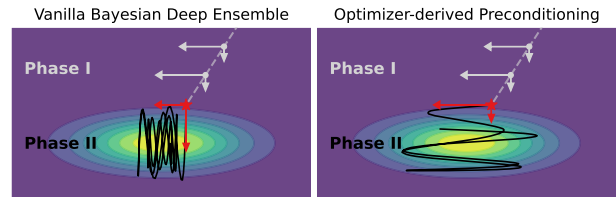


Figure 1: Simulation of optimizer-derived preconditioning in a two-phase Bayesian Deep Ensemble pipeline on an ill-conditioned Gaussian mixture. Phase I shows AdamW optimization trajectories and associated coordinate-wise second-moment statistics (grey arrows). Phase II depicts SGHMC sampling trajectories (black line) and the used preconditioning magnitudes (red arrows). Left: Vanilla sampling with an identity mass matrix explores inefficiently due to strong anisotropy. Right: Optimizer-derived preconditioning aligns the sampler with the local geometry, enabling more efficient exploration and faster mixing.

cost of exploring the complex, high-dimensional, and generally ill-conditioned posterior landscapes typical of deep Bayesian neural networks (BNNs) (Papamarkou et al., 2024).

A recent and growingly popular post-Bayesian approach for scalable inference in BNNs is the two-stage BDE procedure (Sommer et al., 2024, 2026; Paulin et al., 2025; Duffield et al., 2025). This approach first uses standard optimization to find multiple distinct modes akin to Deep Ensembles (Lakshminarayanan et al., 2017) and then initializes independent MCMC samplers from these solutions to explore their local basins of attraction. This strategy has shown strong empirical performance, often outperforming common variational methods (Kobialka et al., 2026). However, the optimization and sampling stages are typically treated as disjoint processes. In such cases, the geometric information about the loss landscape gathered during optimization is discarded, forcing the subsequent sampling phase to rediscover the local curvature from scratch during a potentially long and costly scale-adaptation period.

Our Contributions In this work, we explore a tighter integration of the optimization and sampling phases. We use information of adaptive optimizers like AdamW (Loshchilov and Hutter, 2019), which maintains first- and second-moment estimates derived from gradients as an inexpensive proxy for local curvature. These optimizer-derived statistics can be utilized to directly initialize the preconditioner for subsequent stochastic gradient MCMC (SGMCMC) sampling (Girolami and Calderhead, 2011) (see Figure 1). By providing an informed geometric starting point, this “warmstart” allows the sampler to bypass the potentially lengthy scale-adaptation period (Springenberg et al., 2016), offering a more efficient path to posterior exploration in Bayesian ensembles at a negligible cost. To this end, we

- derive several theoretically grounded ways of using optimizer properties to precondition samplers;
- evaluate multiple optimizer-derived geometry transfer strategies across modalities and architectures;
- assess robustness and the ability to accelerate sampler convergence while measuring the impact on predictive performance and uncertainty quality;
- analyze the local geometry patterns found in realistic BNNs and uncover differences between static and adaptive preconditioning variants.

2 BACKGROUND

In Bayesian deep learning, we aim to infer the posterior distribution $p(\theta|\mathcal{D})$ over a neural network’s parameters. As exact inference is intractable, we rely on approximations such as Stochastic Gradient MCMC (SGMCMC). A recent efficient paradigm is the two-stage *Bayesian Deep Ensemble* (BDE) approach (Sommer et al., 2024; Duffield et al., 2025; Paulin et al., 2025), which initializes independent MCMC chains from high-probability modes found by optimization.

A major challenge in this domain is the efficient exploration of the complex posterior geometry, which often requires preconditioning, for instance through a mass matrix $\mathbf{M}$ in SGHMC. While effective, such preconditioning schemes typically rely on expensive adaptation phases, as in scale-adapted SGHMC (AdaSGHMC; Springenberg et al., 2016). Our work focuses on bridging the optimization and sampling stages by utilizing the curvature estimates (such as those from AdamW) generated during the first stage as a “free,” high-quality preconditioner for the sampling phase.

We provide a comprehensive background on Bayesian inference, BDEs, detailed SGMCMC dynamics, and a full review of related literature in App. A.

3 BRIDGING THE GAP BETWEEN OPTIMIZATION AND SAMPLING

Optimization and sampling both rely on stochastic gradients and auxiliary geometry statistics. While structurally similar, the transfer of terminal optimizer statistics to precondition samplers remains unexplored. We formalize and evaluate strategies to transfer these estimates—ranging from local gradients to smoothed curvature proxies—for efficient posterior exploration.

3.1 Parallels Between Optimizers and Samplers

Let $\theta_t \in \mathbb{R}^d$ denote the parameters at iteration t , and $\mathbf{g}_t := \nabla_{\theta} \hat{\ell}(\theta_t)$ be the stochastic gradient of the negative log-likelihood loss $\hat{\ell}$ evaluated on a mini-batch at time t . Adaptive optimizers such as AdamW maintain exponential moving averages of these gradients:

$$\mu_t = \beta_1 \mu_{t-1} + (1 - \beta_1) \mathbf{g}_{t-1}, \quad (1)$$

$$\nu_t = \beta_2 \nu_{t-1} + (1 - \beta_2) \mathbf{g}_{t-1}^2, \quad (2)$$

where μ_t acts as momentum and ν_t as a curvature proxy (gradient variance), with decay rates $\beta_{1,2} \in (0, 1)$. Similarly, SGMCMC uses a preconditioner $\mathbf{M}$ to rescale updates (see App. A.3). Notably, AdaSGHMC adapts $\mathbf{M}$ using statistics structurally identical to ν_t , utilizing the posterior gradient $\nabla \tilde{U}(\theta)$ rather than $\mathbf{g}_t$.

Local Spectral Alignment In the following, we elaborate on the theoretical bridge between Phase I and II in BDEs, allowing us to reuse information obtained during optimization for the subsequent sampling. In general, a theoretical link between the optimization loss and the posterior landscape follows from the correspondence between regularization and prior choice. For L_2 -regularization with strength $\lambda > 0$, a mathematical equivalence between the regularized negative log-likelihood and the unnormalized log-posterior is obtained by using a Gaussian prior $\theta_t \sim \mathcal{N}(\mathbf{0}, \rho^2 \mathbf{I})$ with $\lambda = (2\rho^2)^{-1}$, yielding $\nabla \tilde{U}(\theta_t) = \mathbf{g}_t + \lambda \theta_t$. Beyond this static alignment, we justify the transfer of geometric information through the following proposition, whose proof is deferred to App. B.1.

Proposition 3.1. *Assume the posterior potential $U(\theta)$ is locally quadratic in a basin $\mathcal{B}(\theta^*)$ around a mode θ^* , such that $U(\theta) \approx \frac{1}{2}(\theta - \theta^*)^\top \mathbf{H}^*(\theta - \theta^*)$ where $\mathbf{H}^* = \nabla^2 U(\theta^*)$ is the Hessian at the mode. If the mass matrix is set to $\mathbf{M} = \mathbf{H}^*$, the preconditioned SGHMC dynamics achieve optimal local mixing by inducing an isotropic Gaussian approximation of the target distribution with a unit condition number.*

Notably, by employing a static mass matrix $\mathbf{M}$ derived at the mode, the dynamics satisfy the constant

fluctuation-dissipation theorem. This allows the system to exploit the local geometry while preserving the correct stationary distribution, without requiring the computationally expensive time-dependent friction terms or Christoffel-symbol corrections typically necessitated by Riemannian Manifold MCMC (Girolami and Calderhead, 2011). Further, in high-dimensional neural networks, the full Hessian $\mathbf{H} = \nabla^2 U(\boldsymbol{\theta})$ is computationally intractable and sampling is usually done using a diagonal mass matrix $\mathbf{M}$. In this setting, the optimizer’s terminal state provides various computationally efficient diagonal proxies for $\mathbf{H}^*$ such as $\boldsymbol{\nu}_T$. For models where the stochastic gradient noise is approximately Gaussian, the second-moment accumulator $\boldsymbol{\nu}_T$ serves as a diagonal estimator of the Fisher Information Matrix (FIM), which coincides with the expected Hessian (Khan et al., 2018). In practice, AdamW-style optimizers utilize root-scaling for numerical stability in non-convex landscapes where curvature may be singular or noisy. Thus, initializing $\mathbf{M} = \text{diag}(\sqrt{\boldsymbol{\nu}_T} + \epsilon \mathbf{I})$ transfers the specific “effective” metric tensor that successfully navigated the optimization landscape to the sampling phase. This construction serves as a regularized spectral surrogate, preserving local geometry while ensuring the dynamics remain well-posed.

3.2 Optimizer-Informed Preconditioning Strategies

We consider diagonal mass matrices $\mathbf{M} = \text{diag}(m_1, \dots, m_d)$ that reuse optimizer statistics

accumulated during training. Beyond the isotropic baseline $\mathbf{M} = \mathbf{I}_d$, we study the following constructions:

$$\begin{aligned} \text{Gradient-based: } \mathbf{M} &= \text{diag}(\mathbf{g}_T^2 + \epsilon \mathbf{1}), \\ \text{First-moment: } \mathbf{M} &= \text{diag}(\boldsymbol{\mu}_T^2 + \epsilon \mathbf{1}), \\ \text{Second-moment: } \mathbf{M} &= \text{diag}(\sqrt{\boldsymbol{\nu}_T} + \epsilon \mathbf{1}). \end{aligned}$$

Here $\mathbf{g}_T, \boldsymbol{\mu}_T, \boldsymbol{\nu}_T$ denote the final gradient, first-moment and second-moment statistics, respectively. This unified formulation highlights that optimizer states readily provide curvature proxies for initializing $\mathbf{M}$.

These strategies differ fundamentally in how they approximate posterior geometry. A preconditioner based on $\mathbf{g}_T$ is purely local, reflecting the stochastic gradient at the final iteration. While this can locally yield highly adaptive scaling, it is also susceptible to noise and batch-to-batch variability, and thus provides only a myopic proxy for steepness. Using $\boldsymbol{\mu}_T$ introduces temporal smoothing: since $\beta_1 < \beta_2$ in practice, the first-moment average emphasizes more recent gradients, yielding a preconditioner that captures local structure but with improved stability. In contrast, $\boldsymbol{\nu}_T$ accumulates squared gradients with usually slower forgetting, effectively approximating parameter-wise Fisher information. This makes the preconditioner less responsive to local fluctuations but more robust and theoretically aligned with variance-based geometry. Following the RMSprop-inspired approach from Li et al. (2016), we scale $\boldsymbol{\nu}_T$ by taking the element-wise square root.

Table 1: Average Performance (3 replications) of layer-wise preconditioned SGHMC (no warmup) and AdaSGHMC (with warmup) across all datasets. Relative improvements over the Deep Ensemble baseline are shown in parentheses. Best values per column are boldfaced. Further details and results in Appendices D.6 and D.7.

(a) Predictive performance is reported as root mean squared error (RMSE $\downarrow$) on regression tasks and Accuracy ($\uparrow$) on classification tasks.

Sampler	Preconditioning	RMSE ($\downarrow$)		Accuracy ($\uparrow$)			Avg. Improv. Over DE
		bike	protein	CIFAR-10	F-MNIST	shakespeare	
Deep Ensemble	–	0.2445	0.6589	0.9019	0.9157	0.5504	–
SGHMC	identity	0.2392 (+2.20%)	0.6533 (+0.86%)	0.9150 (+1.46%)	0.9226 (+0.75%)	0.5536 (+0.58%)	+1.17%
SGHMC	gradient	0.2406 (+1.59%)	0.6492 (+1.47%)	0.9155 (+1.51%)	0.9268 (+1.21%)	0.5533 (+0.53%)	+1.26%
SGHMC	1st moment	0.2405 (+1.64%)	0.6504 (+1.30%)	0.9141 (+1.35%)	0.9256 (+1.08%)	0.5543 (+0.71%)	+1.22%
SGHMC	2nd moment	0.2400 (+1.87%)	0.6470 (+1.81%)	0.9154 (+1.50%)	0.9231 (+0.81%)	0.5535 (+0.56%)	+1.31%
AdaSGHMC	adaptive	0.2424 (+0.87%)	0.6560 (+0.44%)	0.9055 (+0.39%)	0.9197 (+0.43%)	0.5523 (+0.35%)	+0.50%

(b) Uncertainty quantification is reported as log pointwise predictive density (LPPD $\uparrow$) for regression and classification tasks and perplexity ($\downarrow$) for language modeling tasks.

Sampler	Preconditioning	LPPD ($\uparrow$)				Perplexity ($\downarrow$)	Avg. Improv. Over DE
		bike	protein	CIFAR-10	F-MNIST	shakespeare	
Deep Ensemble	–	0.5834	-0.7173	-0.3047	-0.2318	4.3687	–
SGHMC	identity	0.6190 (+6.11%)	-0.6434 (+10.30%)	-0.2641 (+13.3%)	-0.2216 (+4.40%)	4.2645 (+2.38%)	+7.30%
SGHMC	gradient	0.6418 (+10.0%)	-0.6171 (+13.97%)	-0.2583 (+15.3%)	-0.2102 (+9.34%)	4.2846 (+1.92%)	+10.1%
SGHMC	1st moment	0.6416 (+9.99%)	-0.6274 (+12.54%)	-0.2644 (+13.3%)	-0.2118 (+8.64%)	4.2801 (+2.03%)	+9.29%
SGHMC	2nd moment	0.6397 (+9.67%)	-0.6002 (+16.32%)	-0.2615 (+14.2%)	-0.2214 (+4.48%)	4.2894 (+1.81%)	+9.29%
AdaSGHMC	adaptive	0.6182 (+5.98%)	-0.6908 (+3.69%)	-0.2942 (+3.45%)	-0.2197 (+5.25%)	4.3382 (+0.70%)	+3.81%

3.3 Layer-wise Preconditioning

To improve numerical stability, we also evaluate a *layer-wise* strategy (Wenzel et al., 2020). Instead of maintaining a value for every parameter, we compute a single scalar for each layer ℓ by averaging the optimizer statistics of its parameters and normalizing across layers; the precise formulation is given in App. B.2. This reduces granularity but acts as a regularizer against extreme single-parameter values and upper bounds the maximum parameter-wise step size, increasing stability.

4 EMPIRICAL STUDY

We evaluate optimizer-derived preconditioning on tabular regression (bikesharing, protein), image classification (F-MNIST, CIFAR-10), and language modeling (shakespeare) using FCNs, CNNs, ResNets, and Transformers. We compare our preconditioned SGHMC (no warmup) against Deep Ensembles and AdaSGHMC (with warmup) (see App. C for the full setup).

4.1 Performance and Efficiency

As shown in Table 1, optimizer-derived preconditioning consistently outperforms the Deep Ensemble baseline across all tasks and metrics. Notably, it also surpasses AdaSGHMC, particularly in uncertainty quantification (LPPD, Perplexity), despite using zero sampling warmup. When compared to Vanilla SGHMC, an optimistic baseline that, due to higher hyperparameter sensitivity, benefits most from reporting peak performance over the same uniform step-size sweep applied to all methods, optimizer-derived preconditioning performs on par or better across all datasets. While identity preconditioning already improves over DEs, incorporating optimizer-derived geometry yields additional and consistent gains, most notably for UQ on tabular and image data. We also analyzed the impact of warmup duration (see Figure 2; App. D.1). Local strategies (Gradient, 1st-Moment) achieve peak performance immediately (with zero warmup), making them ideal for budget-constrained settings. The 2nd-Moment strategy benefits from a longer warmup, eventually yielding the highest overall performance by leveraging global curvature information. This allows for a significant reduction in computational budget while maintaining or exceeding baseline performance. A dedicated discussion on runtime savings is provided in the App. D.3.

4.2 Robustness and Stability

Optimizer-informed preconditioning significantly enhances robustness to the step size η . While vanilla SGHMC degrades sharply with suboptimal η , our preconditioned variants maintain stable performance

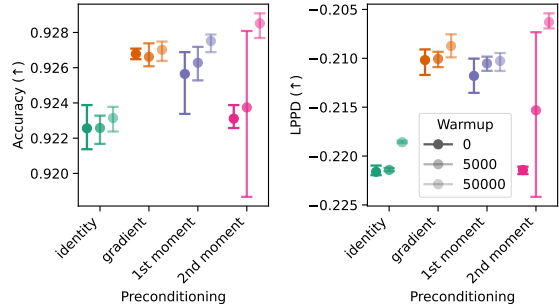


Figure 2: Minimal, maximal, and average performance on the F-MNIST dataset for different warmup lengths after warmstarting with AdamW and sampling with (preconditioned) SGHMC.

across a broad range. The layer-wise normalization (Section 3) acts as a safeguard, rescaling effective updates to prevent discretization error (see App. D.2). We further compared parameter-wise vs. layer-wise granularity and found the layer-wise approach to be substantially more numerically stable without compromising predictive performance (see App. D.7).

4.3 Analysis of Adapted Geometry

We investigated why the static transfer outperforms the adaptive AdaSGHMC. By analyzing the cosine similarity of mass matrices (see App. D.5), we found that AdaSGHMC’s internal adaptation tends to “overwrite” the initial geometry, making the chains in the ensemble highly self-similar. In contrast, our static optimizer-derived preconditioning preserves the distinct geometric information of each mode found in Phase I. This preservation of ensemble diversity likely accounts for the superior generalization performance. An analysis of empirical preconditioning distributions that additionally supports the layer-wise approach is provided in App. D.4.

5 DISCUSSION

Our study shows that transferring optimizer-derived geometry to an SGMCMC sampler is an effective, zero-cost strategy for improving BDEs. This approach consistently retains and often enhances predictive performance and robustness, while reducing computational cost by eliminating the warmup phase. Our results suggest using gradient or first-moment preconditioning under constrained budgets, while second-moment estimates yield superior results given longer runs. Our ablations confirm that layer-wise preconditioning is more numerically stable than parameter-wise counterparts. We also identified that adaptive samplers overwrite the transferred geometry, reducing ensemble diversity and providing evidence for their inferior performance.

References

- Adilova, L., Andriushchenko, M., Kamp, M., Fischer, A., and Jaggi, M. (2024). Layer-wise linear mode connectivity. In *The Twelfth International Conference on Learning Representations*.
- Becker, S. and Cun, Y. L. (1989). Improving the convergence of back-propagation learning with second order methods. In *Proceedings of the 1988 Connectionist Models Summer School*, pages 29–37. San Francisco, CA: Morgan Kaufmann.
- Bradbury, J., Frostig, R., Hawkins, P., Johnson, M. J., Leary, C., Maclaurin, D., Necula, G., Paszke, A., VanderPlas, J., Wanderman-Milne, S., and Zhang, Q. (2018). JAX: composable transformations of Python+NumPy programs.
- Cabezas, A., Corenflos, A., Lao, J., and Louf, R. (2024). Blackjax: Composable Bayesian inference in JAX.
- Chen, T., Fox, E., and Guestrin, C. (2014). Stochastic Gradient Hamiltonian Monte Carlo. In *Proceedings of the 31st International Conference on Machine Learning*, pages 1683–1691. PMLR.
- Dua, D. and Graff, C. (2017). UCI machine learning repository.
- Duffield, S., Donatella, K., Chiu, J., Klett, P., and Simpson, D. (2025). Scalable bayesian learning with posteriors. In *The Thirteenth International Conference on Learning Representations*.
- Fanaee-T, H. (2013). Bike Sharing Dataset. UCI Machine Learning Repository.
- Girolami, M. and Calderhead, B. (2011). Riemann manifold Langevin and Hamiltonian Monte Carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 73(2):123–214.
- Izmailov, P., Vikram, S., Hoffman, M. D., and Wilson, A. G. (2021). What Are Bayesian Neural Network Posteriors Really Like? In *Proceedings of the 38th International Conference on Machine Learning*, pages 4629–4640. PMLR.
- Karpathy, A. (2022). NanoGPT. <https://github.com/karpathy/nanoGPT>.
- Khan, M., Nielsen, D., Tangkaratt, V., Lin, W., Gal, Y., and Srivastava, A. (2018). Fast and scalable bayesian deep learning by weight-perturbation in adam. In *International conference on machine learning*, pages 2611–2620. PMLR.
- Kobialka, J., Sommer, E., Kolb, C., Kwon, J., Dold, D., and Rügamer, D. (2026). On the Interplay of Priors and Overparametrization in Bayesian Neural Network Posteriors. In *Proceedings of the 29th International Conference on Artificial Intelligence and Statistics*.
- Krizhevsky, A., Hinton, G., et al. (2009). Learning multiple layers of features from tiny images.
- Lakshminarayanan, B., Pritzel, A., and Blundell, C. (2017). Simple and scalable predictive uncertainty estimation using deep ensembles. *Advances in neural information processing systems*, 30.
- Li, C., Chen, C., Carlson, D., and Carin, L. (2016). Preconditioned Stochastic Gradient Langevin Dynamics for Deep Neural Networks. *Proceedings of the AAAI Conference on Artificial Intelligence*, 30(1).
- Li, Y. X., Dangel, F., Tam, D., and Raffel, C. (2025). Fishers for Free? Approximating the Fisher Information Matrix by Recycling the Squared Gradient Accumulator. In *Proceedings of the 42nd International Conference on Machine Learning*, Vancouver, Canada.
- Liu, H., Li, Z., Hall, D. L. W., Liang, P., and Ma, T. (2024). Sophia: A scalable stochastic second-order optimizer for language model pre-training. In *The Twelfth International Conference on Learning Representations*.
- Loshchilov, I. and Hutter, F. (2019). Decoupled Weight Decay Regularization. In *The Seventh International Conference on Learning Representations*, New Orleans.
- Ma, Y.-A., Chen, T., and Fox, E. B. (2015). A complete recipe for stochastic gradient MCMC. In *Proceedings of the 29th International Conference on Neural Information Processing Systems*, volume 2, pages 2917–2925, Cambridge, MA, USA. MIT Press.
- Papamarkou, T. (2023). Approximate blocked Gibbs sampling for Bayesian neural networks. *Statistics and Computing*, 33.
- Papamarkou, T., Skoularidou, M., Palla, K., Aitchison, L., Arbel, J., Dunson, D., Filippone, M., Fortuin, V., Hennig, P., Hernández-Lobato, J. M., Hubin, A., Immer, A., Karaletsos, T., Khan, M. E., Kristiadi, A., Li, Y., Mandt, S., Nemeth, C., Osborne, M. A., Rudner, T. G. J., Rügamer, D., Teh, Y. W., Welling, M., Wilson, A. G., and Zhang, R. (2024). Position: Bayesian Deep Learning is Needed in the Age of Large-Scale AI. In *Proceedings of the 41st International Conference on Machine Learning*, pages 39556–39586. PMLR.

- Paulin, D., Whalley, P. A., Chada, N. K., and Leimkuhler, B. J. (2025). Sampling from Bayesian Neural Network Posteriors with Symmetric Minibatch Splitting Langevin Dynamics. In *The 28th International Conference on Artificial Intelligence and Statistics*.
- Rundel, D., Sommer, E., Bischl, B., Rügamer, D., and Feurer, M. (2025). Efficiently Warmstarting MCMC for BNNs. In *Frontiers in Probabilistic Inference: Learning meets Sampling*.
- Shen, Y., Daheim, N., Cong, B., Nickl, P., Marconi, G. M., Bazan, C., Yokota, R., Gurevych, I., Cremers, D., Khan, M. E., and Möllenhoff, T. (2024). Variational learning is effective for large deep networks. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235, pages 44665–44686, Vienna, Austria.
- Singh, S. and Krishnan, S. (2020). Filter response normalization layer: Eliminating batch dependence in the training of deep neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11237–11246.
- Sommer, E., Diao, K., Robnik, J., Seljak, U., and Rügamer, D. (2026). Can microcanonical langevin dynamics leverage mini-batch gradient noise?
- Sommer, E., Robnik, J., Nozadze, G., Seljak, U., and Rügamer, D. (2025). Microcanonical Langevin Ensembles: Advancing the Sampling of Bayesian Neural Networks. In *The Thirteenth International Conference on Learning Representations*.
- Sommer, E., Wimmer, L., Papamarkou, T., Bothmann, L., Bischl, B., and Rügamer, D. (2024). Connecting the Dots: Is Mode-Connectedness the Key to Feasible Sample-Based Inference in Bayesian Neural Networks? In *Proceedings of the 41st International Conference on Machine Learning*, pages 45988–46018. PMLR.
- Springenberg, J. T., Klein, A., Falkner, S., and Hutter, F. (2016). Bayesian Optimization with Robust Bayesian Neural Networks. In *Advances in Neural Information Processing Systems*, volume 29. Curran Associates, Inc.
- Wenzel, F., Roth, K., Veeling, B. S., Swiatkowski, J., Tran, L., Mandt, S., Snoek, J., Salimans, T., Jenatton, R., and Nowozin, S. (2020). How good is the bayes posterior in deep neural networks really? In *Proceedings of the 37th International Conference on Machine Learning*, volume 119, pages 10248–10259. JMLR.
- Xiao, H., Rasul, K., and Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms.
- Yao, Z., Gholami, A., Shen, S., Mustafa, M., Keutzer, K., and Mahoney, M. (2021). Adahessian: An adaptive second order optimizer for machine learning. In *proceedings of the AAAI conference on artificial intelligence*, volume 35, pages 10665–10673.
- Zhang, R., Li, C., Zhang, J., Chen, C., and Wilson, A. G. (2020). Cyclical Stochastic Gradient MCMC for Bayesian Deep Learning. In *The Eighth International Conference on Learning Representations*.

A EXTENDED BACKGROUND & RELATED WORK

A.1 Bayesian Inference for Neural Networks

In Bayesian deep learning, we aim to infer the posterior distribution over a neural network’s parameters $\theta \in \mathbb{R}^d$ given a dataset $\mathcal{D} = \{(y_i, \mathbf{x}_i)\}_{i=1}^N$. According to Bayes’ rule, the posterior is $p(\theta|\mathcal{D}) \propto p(\mathcal{D}|\theta)p(\theta)$, where $p(\mathcal{D}|\theta)$ is the likelihood and $p(\theta)$ is the prior. Throughout, we assume isotropic Gaussian priors on θ (Kobialka et al., 2026). As this posterior is intractable for modern network architectures, we rely on approximation methods. Sampling-based approaches, such as SGMCMC, are a powerful class of methods that can asymptotically recover the true posterior without the restrictive assumptions of variational inference (Izmailov et al., 2021).

A major challenge for SGMCMC is efficiently exploring the high-dimensional, multi-modal posteriors of modern neural networks. A recently proposed remedy is a two-stage approach (Sommer et al., 2024, 2026; Duffield et al., 2025; Paulin et al., 2025), also referred to as *Bayesian Deep Ensemble*. It first uses conventional optimization to find multiple distinct, high-probability regions (modes) in the parameter space. Subsequently, independent MCMC chains are initialized from these modes to explore the local posterior geometry. Prior work has investigated optimal BDE chain initialization under constrained computational budgets (Rundel et al., 2025). Our work focuses on creating a more synergistic link between these optimization and sampling stages, which, so far, have mostly been treated as independent components in the literature.

A.2 Related Methods

In recent years, various approaches have been proposed to connect optimization and sampling and use synergies between both, including examples such as tempering schedules (Duffield et al., 2025) or repeated cycles that alternate between optimization-based exploration and sampled exploitation (Zhang et al., 2020). In this paper, we focus on the discrete two-stage approach implemented by BDEs due to its modularity and wide applicability, including, for example, the use of pre-trained models or publicly available optimization recipes. A different yet related research question in the field of sampling-based inference for BNNs is whether geometric adaptation of sampler dynamics can boost their efficiency. Despite foundational work motivating the use of preconditioning to navigate complex posteriors (Girolami and Calderhead, 2011; Ma et al., 2015; Wenzel et al., 2020), its adoption in practice has been notably sparse. As such, prominent recent work often forgoes preconditioning entirely (see e.g. Sommer et al., 2025), and popular libraries like **posteriors** (Duffield et al., 2025) currently lack support for preconditioned sampling. This reluctance may stem from the practical challenges of determining and adapting the preconditioner, which can introduce its own set of hyperparameters or might require an extended warmup phase for scale-adaption (Springenberg et al., 2016).

A.3 Preconditioned Stochastic Gradient MCMC

Stochastic gradient Hamiltonian Monte Carlo (SGHMC) is often considered the state-of-the-art SGMCMC algorithm. It introduces an auxiliary momentum variable $\mathbf{r}$ to encourage more efficient exploration of the target distribution (Chen et al., 2014). Its dynamics are governed by a mass matrix $\mathbf{M}$, which can be viewed as a *preconditioner* that adapts the sampler’s steps to the local geometry of the posterior. The discretized update equations are

$$\theta_{t+1} = \theta_t + \eta \mathbf{M}^{-1} \mathbf{r}_t, \tag{3}$$

$$\mathbf{r}_{t+1} = \mathbf{r}_t - \eta \nabla \tilde{U}(\theta_t) - \eta \mathbf{C} \mathbf{M}^{-1} \mathbf{r}_t + \sqrt{2\eta(\mathbf{C} - \hat{\mathbf{E}})} \mathbf{Z}_t, \tag{4}$$

where η is the step size, $\mathbf{C}$ is a friction matrix, $\nabla \tilde{U}(\theta) = -\nabla \log p(\theta) - \frac{N}{|\mathcal{B}|} \sum_{(\mathbf{x}, y) \in \mathcal{B}} \nabla \log p(y|\mathbf{x}, \theta)$ is the stochastic gradient of the unnormalized log-posterior based on mini-batch $\mathcal{B} \subset \mathcal{D}$, $\hat{\mathbf{E}}$ is an estimate of the gradient noise covariance, and $\mathbf{Z}_t \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

The choice of $\mathbf{M}$ can have a considerable effect on the sampling effectiveness. A sampler using an isotropic mass matrix ($\mathbf{M} \propto \mathbf{I}$) potentially struggles in ill-conditioned landscapes where curvature varies drastically across dimensions (Girolami and Calderhead, 2011), leading to inefficient exploration (see Figure 1, left). A well-chosen preconditioner reshapes the geometry, enabling larger, more effective steps (see Figure 1, right). To automate this, methods like scale-adapted SGHMC (AdaSGHMC; Springenberg et al., 2016) dynamically adapt a diagonal

mass matrix $\mathbf{M} = \text{diag}(\hat{\mathbf{v}}^{1/2})$ during a warmup phase, where $\hat{\mathbf{v}}$ is a running estimate of the squared gradient of $\tilde{U}(\boldsymbol{\theta})$. However, this adaptation period can be lengthy and adds computational overhead.

A.4 Curvature Estimation in Adaptive Optimizers

Modern adaptive optimizers are designed to handle the ill-conditioned loss landscapes of deep networks. They do so by maintaining running estimates of local curvature, which are readily available as a free byproduct upon completion of the optimization phase.

One of the most frequently used optimizers nowadays, *AdamW* (Loshchilov and Hutter, 2019), maintains both a first moment vector $\boldsymbol{\mu}_t$ and a second moment vector, $\boldsymbol{\nu}_t$, which are both exponential moving average functions of the gradients. In the optimizer, the latter serves as a diagonal preconditioner for the optimization step, adaptively rescaling the learning rate for each parameter. Various other optimizers also either maintain an estimate of second-order information or directly store this information in every iteration (Becker and Cun, 1989; Yao et al., 2021; Liu et al., 2024). Similar approaches exist in variational learning. One example is the optimizer IVON (Shen et al., 2024), which follows a similar idea by maintaining an online estimate of the diagonal of the (expected) Hessian to approximate the posterior variance for variational inference. Our proposed optimizer-informed preconditioning is agnostic to the specific training algorithm and applies to any method that maintains curvature-related statistics.

A.5 Theoretical Connections: Fisher Information

The idea of using geometry to guide sampling is theoretically well-established, with foundational work on Riemannian Manifold MCMC demonstrating significant efficiency gains by defining a metric tensor based on the Fisher Information Matrix (FIM; Girolami and Calderhead, 2011). Thereby, the FIM is mostly considered the theoretically optimal choice of preconditioner as it fully captures the local geometry of the likelihood (Ma et al., 2015). However, computing the full FIM is intractable for modern architectures.

A critical insight is that the diagonal of the FIM can be cheaply approximated. Recent work has established a strong connection between the statistics kept by adaptive optimizers and the diagonal FIM, a concept sometimes termed “Fisher for free” (Li et al., 2025). Specifically, the squared gradient accumulator from optimizers like Adam serves as a computationally trivial approximation to the empirical FIM diagonal. While this connection is known, it has not been systematically studied as a mechanism to bridge the optimization and sampling stages in the BDE context. Most existing preconditioning methods for SGMCMC either rely on costly online adaptation during warmup (Springenberg et al., 2016) or require periodic re-estimation (Wenzel et al., 2020). Our work investigates a simpler, more direct approach: leveraging the final state of the optimizer’s curvature estimate as a static, high-quality preconditioner for the entire subsequent sampling run, thereby aiming to reduce or eliminate the adaptive warmup phase entirely.

A.6 Applicability to Non-Gradient Samplers

While our primary evaluation focuses on the kinetic dynamics of SGHMC, the principle of optimizer-derived geometric transfer extends naturally to non-gradient-based samplers, such as Gibbs sampling. In high-dimensional Bayesian neural networks, blocked Gibbs samplers often suffer from vanishing acceptance rates due to isotropic proposal distributions that fail to account for parameter correlations (Papamarkou, 2023). By repurposing the second-moment proxies like $\boldsymbol{\nu}_T$ as the scales of an anisotropic proposal density, one could transform the standard spherical proposals into ellipsoids aligned with the local posterior curvature. This geometric alignment would allow the sampler to take larger, more effective steps without the associated penalty in rejection rates, essentially solving the “vanishing acceptance” problem through an inexpensive geometric warmstart rather than complex block-size reduction strategies.

B PROOFS AND METHOD DETAILS

B.1 Proof of Proposition 3.1

Proof. The SGHMC dynamics are governed by the Hamiltonian $H(\boldsymbol{\theta}, \mathbf{r}) = U(\boldsymbol{\theta}) + \frac{1}{2}\mathbf{r}^\top \mathbf{M}^{-1}\mathbf{r}$. To analyze the efficiency of the sampling trajectory, we consider the geometry of the potential within the local basin $\mathcal{B}(\theta^*)$ (Paulin et al., 2025). We ignore stochastic gradient noise and friction terms, which do not affect the local spectral argument

given in the following. Under the local Gaussian approximation, the potential energy is $U(\boldsymbol{\theta}) \approx \frac{1}{2} \|\boldsymbol{\theta} - \boldsymbol{\theta}^*\|_{\mathbf{H}^*}^2$. Using the linear coordinate transformation $\tilde{\boldsymbol{\theta}} = \mathbf{M}^{1/2}(\boldsymbol{\theta} - \boldsymbol{\theta}^*)$, the potential in the transformed space becomes:

$$\begin{aligned}\tilde{U}(\tilde{\boldsymbol{\theta}}) &= \frac{1}{2} (\mathbf{M}^{-1/2} \tilde{\boldsymbol{\theta}})^\top \mathbf{H}^* (\mathbf{M}^{-1/2} \tilde{\boldsymbol{\theta}}) \\ &= \frac{1}{2} \tilde{\boldsymbol{\theta}}^\top (\mathbf{M}^{-1/2} \mathbf{H}^* \mathbf{M}^{-1/2}) \tilde{\boldsymbol{\theta}}.\end{aligned}$$

Substituting the initialization $\mathbf{M} = \mathbf{H}^*$ into the transformed Hessian $\tilde{\mathbf{H}} = \mathbf{M}^{-1/2} \mathbf{H}^* \mathbf{M}^{-1/2}$ yields $\tilde{\mathbf{H}} = (\mathbf{H}^*)^{-1/2} \mathbf{H}^* (\mathbf{H}^*)^{-1/2} = \mathbf{I}$. The transformed potential is thus $\tilde{U}(\tilde{\boldsymbol{\theta}}) = \frac{1}{2} \|\tilde{\boldsymbol{\theta}}\|^2$, which is isotropic. Consequently, the condition number $\kappa(\tilde{\mathbf{H}}) = \lambda_{\max}/\lambda_{\min} = 1$, minimizing the localized mixing time and removing anisotropy as a source of poor local mixing. $\square$

B.2 Details on Layer-wise Normalization

Inspired by work suggesting that parameters within a neural network layer share statistical properties (Sommer et al., 2024; Adilova et al., 2024), we evaluate a layer-wise preconditioning strategy. This approach, following Wenzel et al. (2020), aggregates the parameter-wise statistics to assign a single preconditioning value to all parameters within a given layer or parameter group (e.g. kernels and biases).

Let $\mathbf{m} = (m_1, \dots, m_d)$ be the vector of parameter-wise preconditioning values derived from one of the methods above. For each of the L layers, we compute a single scalar value by averaging the values of its constituent parameters

$$m^{(\ell)} = \frac{1}{|\mathcal{I}_\ell|} \sum_{i \in \mathcal{I}_\ell} m_i, \quad (5)$$

where $\mathcal{I}_\ell$ is the set of indices for parameters in layer ℓ . To make the overall scale of the preconditioner independent of the step size η , we normalize these values: $\tilde{m}^{(\ell)} = m^{(\ell)} / (\min_{\ell'=1, \dots, L} m^{(\ell')})$. The final layer-wise mass matrix $\mathbf{M}_{\text{layer}}$ is then constructed by assigning the normalized scalar $\tilde{m}^{(\ell)}$ to all parameters in layer ℓ . As teased in above, this reduces the granularity of the preconditioner but may offer a more stable and robust alternative, since the normalization naturally upper bounds the maximum parameterwise step size to η , while allowing smaller updates if necessary.

C EXPERIMENTAL SETUP & FURTHER DETAILS

Software and Computing Environment Our implementation is built in Python using `jax` (Bradbury et al., 2018) and `BlackJAX` (Cabezas et al., 2024) and builds on the codebase of Kobialka et al. (2026). We ran our experiments on NVIDIA A100 and NVIDIA RTX A6000 GPUs. A reproducible codebase is available at https://github.com/EmanuelSommer/geopost_optimal2026.

Computational Cost Runtime scales with model size and dataset complexity. For the LeNet architecture on F-MNIST, warmstarting across 4 parallel chains requires 1–1.5 hours, while the subsequent sampling phase takes up to 1 hour in the most demanding configuration (50,000 warmup steps, 100,000 samples). Our experimental grid for F-MNIST consists of 270 runs (3 seeds $\times$ 3 warmup lengths $\times$ 3 step sizes $\times$ 5 preconditioning strategies $\times$ 2 granularities), totaling approximately 500 GPU hours for this dataset alone. CIFAR-10 experiments exhibited similar computational demands. In contrast, the smaller FCN models and the nanoGPT architecture required only approximately 1/4 of the time per run. Across all experiments, the total computational cost amounts to approximately 1,750 GPU hours on NVIDIA A100 and RTX A6000 hardware.

Datasets Our empirical evaluation is conducted on a diverse suite of benchmark datasets, detailed in Table 2. For tabular regression tasks, we partitioned the data into 70% for training, 10% for validation, and 20% for testing. For the image classification benchmarks, we utilized the official train and test sets. Before training the nanoGPT model, we translated the *tinysakespeare* dataset available from Karpathy (2022) into more modern English using Gemini 2.5 Pro to facilitate a more accessible assessment of the quality of the generated text. This was done using the prompt “Please translate the attached file into simplified modern English. Keep

the structure of the text as is (new line for each speaker, speaker name followed by a colon, then the sentence in a new line)”, together with an attached .txt file of the original text. We call this dataset *modern-shakespeare* and provide it within our public code repository for reproducibility. This adaptation allows for a more intuitive assessment of the generated text’s quality.

Table 2: Overview of the used datasets.

Dataset	Size	Features	Reference
bikesharing	17379	13	Fanace-T (2013)
protein	45730	9	Dua and Graff (2017)
F(ashion)-MNIST	60000	28x28	Xiao et al. (2017)
CIFAR-10	60000	3x32x32	Krizhevsky et al. (2009)
(modern-)shakespeare	39890	65	adapted from Karpathy (2022)

Models Since we are performing our experiments on different datasets and modalities, we also use different architectures throughout. In Table 3 we provide a list of all different models together with their parameter count and the dataset we applied the model onto. The parameter count covers a relatively wide range to test the applicability of preconditioning on highly different model sizes.

Table 3: Overview of the used models.

Model	#Params	Dataset
FCN	10k	bikesharing, protein
LeNet (CNN)	60k	F-MNIST
ResNet-7	428k	CIFAR-10
nanoGPT	10.8M	shakespeare

For the tabular regression tasks on the bikesharing and protein datasets, we employ a Fully Connected Network (FCN) consisting of six hidden layers.

On the F-MNIST dataset, we use a classic LeNet architecture. It features two convolutional layers with 6 and 16 filters, followed by three fully-connected layers. Average pooling is applied after each convolutional block.

For image classification on CIFAR-10 we use a tailored ResNet-7 comprising 428k parameters. Instead of BatchNorm, we employ Filter Response Normalization (FRN) (Singh and Krishnan, 2020), motivated by known limitations of BatchNorm in sampling-based settings (Wenzel et al., 2020; Shen et al., 2024). The full specification of the architecture is provided in Table 4.

Table 4: The Custom ResNet-7 Architecture with 428k trainable parameters. The output shape is specified for a sample input tensor of size $3 \times 32 \times 32$. All convolutional layers use a 3×3 kernel, stride 1, and ‘SAME’ padding, and are followed by Filter Response Normalization (FRN; Singh and Krishnan, 2020).

Stage	Layer Operation(s)	Filters
input	Image	-
stem	Conv-FRN	32
body	Conv-FRN $\rightarrow$ MaxPool	64
	Conv-FRN	64
	Conv-FRN $\rightarrow$ MaxPool	128
	Conv-FRN $\rightarrow$ MaxPool	128
	(Identity Shortcut from previous output)	128
res_block	$\hookrightarrow$ Conv-FRN	128
	$\hookrightarrow$ Add	128
		128
head	Global Average Pool	-
	Fully Connected	-

The setup for character-level modeling on the shakespeare dataset, is as follows. We use a 6-layer, 6-head GPT-style transformer with context length 256 and embedding dimension 384, adapted from Karpathy (2022). Dropout is disabled. The learning rate during warmstarting decays linearly from $3e-4$ to $2.5e-4$.

Sampling Configuration For each distinct experimental configuration, we conducted 3 runs with different random seeds. For SGHMC, we explored the different preconditioning strategies: identity, gradient-based, and 1st and 2nd moment-based both parameter- and layer-wise. We run parallel chains thinned by a factor of 100 using normal isotropic priors. The specific hyperparameter settings for each dataset are detailed in Table 5. AdaSGHMC was evaluated with the same configurations, but the zero warmup case was excluded, as it would be identical to identity-preconditioned SGHMC. In the main results tables, we report its performance with 5000 warmup steps for all datasets except for the shakespere dataset where we use 500 warmup steps.

Table 5: Sampler configurations for each dataset. The best performing step size per dataset, used in the main results tables, are boldfaced.

Dataset	Batch Size	Warmup Steps	Step Sizes	Chains
bikesharing	128	0, 5000, 50000	0.0001 , 0.001, 0.01	8
protein	128	0, 5000, 50000	0.0001 , 0.001, 0.01	8
F-MNIST	256	0, 5000, 50000	0.0001, 0.001 , 0.01	10
CIFAR-10	518	0, 5000, 50000	0.0001 , 0.001, 0.01	8
shakespeare	128	0, 500, 5000	0.0001, 0.0002 , 0.001	4

Tuning Warmstarting Optimizers Prior to running the main experimental grid, we conducted a hyperparameter (HP) tuning study to determine which configuration of the warmstarting optimizers we will use throughout our experiments. This preliminary investigation prevents suboptimal warmstarting configurations from influencing the final results. We tuned two key parameters of AdamW with three discrete values each, yielding this grid:

- **Weight Decay** (λ): {0.001, 0.01, 0.02}
- **Learning Rate** (α): {0.0001, 0.001, 0.01}

The HP configurations were evaluated using both accuracy and LPPD, with the optimal configuration selected based on the best performance across both measures. In our experiments, configurations that maximized accuracy also yielded the highest LPPD value, therefore leaving the choice of the best combination straightforward.

The results across the 3×3 HP grid are shown in Figure 3. The contour plots display performance surfaces created through linear interpolation between grid points. The marginal plots on each axis show the averaged performance along each parameter dimension. The color gradients indicate performance levels, with warmer colors representing lower performance and cooler colors indicating superior performance. The black dots mark the grid points where we evaluated the performance.

The HP tuning study yielded the following optimal configurations, which are used throughout all experiments in this work: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\alpha = 0.01$, $\lambda = 0.02$.

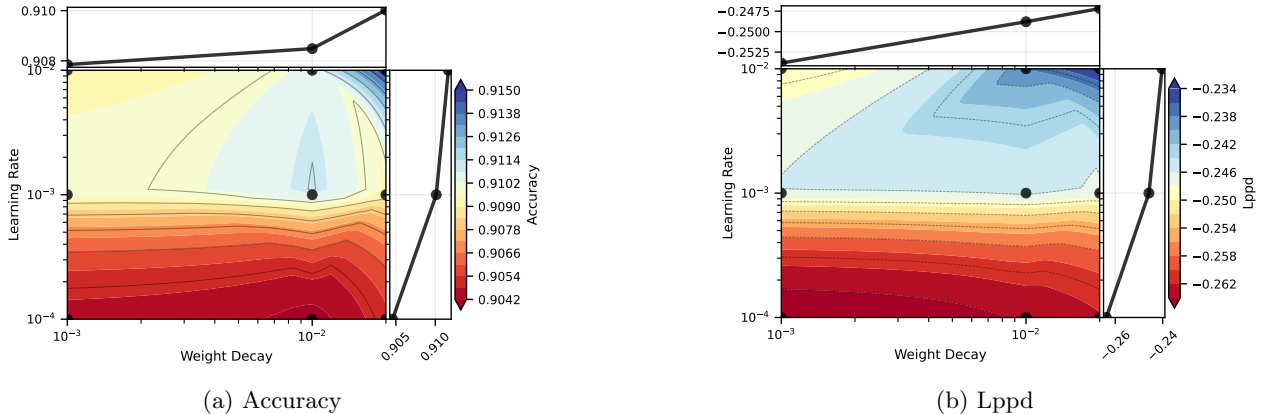


Figure 3: Impact of the HP choice on the warmstarting performance of AdamW for F-MNIST.

D ADDITIONAL EMPIRICAL ANALYSIS

D.1 Warmup and Runtime Analysis

Among the proposed methods, gradient- and first-moment-based preconditioning emerge as the best choice when performing zero warmup, particularly in terms of log pointwise predictive density (LPPD). As illustrated in Figures 2 and 4, these local, geometry-informed strategies achieve peak performance immediately and do not benefit from an extended warmup period. The performance only improves minimal despite the sampler given a much higher sampling budget. In stark contrast, the performance of second-moment preconditioning improves substantially with a longer warmup. We hypothesize this is because the second-moment statistic captures more global, richer geometric information about the posterior landscape, which requires a longer sampling phase to be fully exploited.

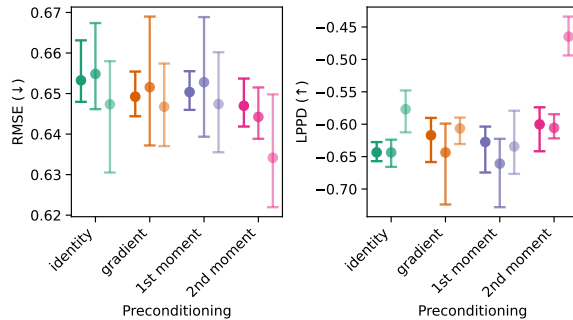


Figure 4: Minimal, maximal, and average performance on the protein dataset for different warmup lengths after warmstarting with AdamW and sampling with SGHMC using the best step size per dataset.

This finding highlights a trade-off in computational budget choice, which influences the preferred preconditioning method. Extended sampling runs (see Figure 6) confirm that these performance differences are stable, with each method converging to a distinct performance level based on the chosen warmup. Consequently, gradient and first-moment methods are preferable for budget-constrained scenarios, while second-moment preconditioning may offer superior performance if a longer warmup phase can be afforded.

D.2 Numerical Robustness

As shown in Figure 5, vanilla SGHMC performance degrades sharply with misspecified step sizes, while preconditioned variants maintain stable performance across a significantly wider range. This robustness stems from the distinction between the user-defined global η and the parameter-specific effective step size ($\eta \cdot m_i^{-1}$). Specifically, the layer-wise normalization (Section 3) acts as a numerical safeguard: even with a large global η intended to accelerate convergence, effective updates are rescaled to remain within a regime where discretization bias is controlled.

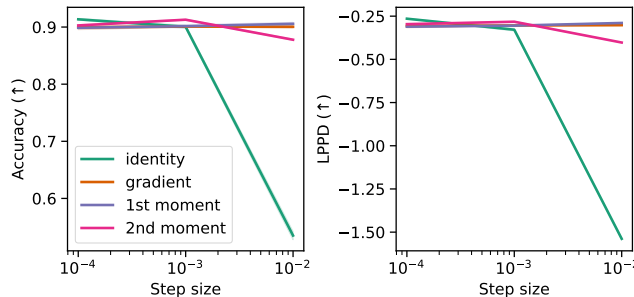


Figure 5: Robustness on CIFAR-10: Preconditioned variants maintain high Accuracy (left) and LPPD (right) across a wider range of step sizes compared to vanilla SGHMC.

This mitigates the risk of performance collapse from suboptimal hyperparameters, aiding practical usability.

While consistent across datasets, we note that for excessively large global step sizes, effective updates eventually exceed local basin bounds, leading to non-convergence for all methods.

D.3 Influence on Runtime and Sampling Budgets

A key advantage of our optimizer-informed preconditioning is the potential for substantial reductions in total sampling budget. As shown in Figure 6, samplers achieve peak performance very early in the sampling run, with accuracy curves quickly flattening. This effect is pronounced even with zero warmup (left plot), making lengthy warmup phases unnecessary and allowing for complete elimination of the warmup period, as demonstrated by our main results in Table 1.

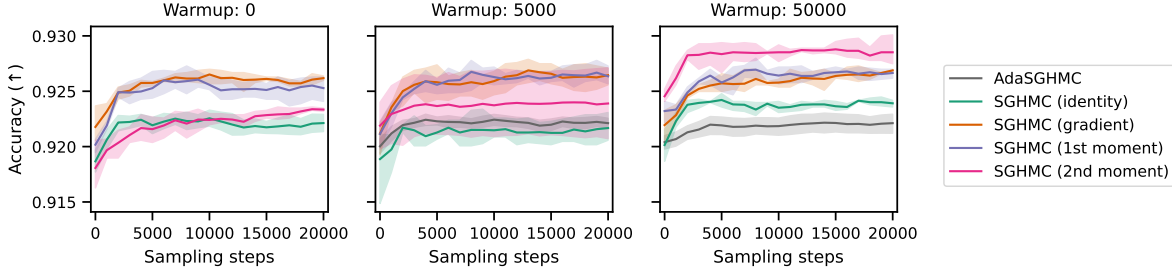


Figure 6: Running mean of the predictive performance with std. dev. (shaded) during the sampling phase after warmstarting with AdamW and for different warmup lengths on F-MNIST.

This efficiency is most pronounced for local preconditioning strategies (gradient and first-moment), which achieve nearly identical performance trajectories irrespective of warmup duration, making them ideal for rapid, low-budget inference. For a pipeline with 5,000 warmup and 10,000 sampling steps—typical for BDE settings—our approach can eliminate the warmup phase, reducing total computational cost by 33% while enabling performance gains. Conversely, if larger computational budgets permit long warmup, the advantage shifts towards second-moment preconditioning, which can leverage the more globally attained geometry estimate to ultimately outperform all other configurations.

D.4 Empirical Preconditioning Distributions

We also investigate the distributions of the preconditioning values in the $\mathbf{M}$ matrix over all chains to understand how they adapt to the network structure. Figure 7 reveals consistent layer-wise patterns across all methods for the LeNet architecture: the first and last layers systematically receive higher preconditioning values, which reduces the effective step size for their parameters and is in line with recent findings on layer-wise exploration of high-performance full-batch samplers (Sommer et al., 2024). These distinct, non-uniform patterns, which are particularly pronounced for second-moment preconditioning, lend strong empirical support to the layer-wise simplification. Furthermore, we observe that gradient- and first-moment preconditioning distributions are nearly identical. This explains the similar performance metrics reported for these two methods when comparing the relative improvements over the different datasets.

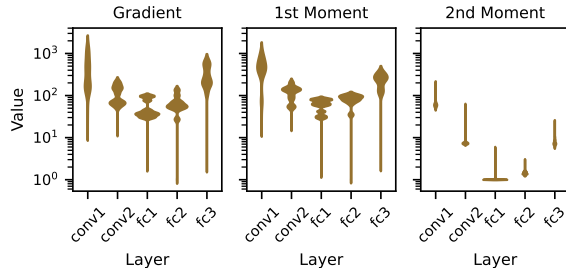


Figure 7: Layer-wise distributions of parameter values in the preconditioning matrix after warmstarting with AdamW on LeNet.

D.5 Adaptive Preconditioning Variants

Interestingly, we find that the positive influence of our geometry transfer above does not translate to SGHMC variants with scale-adaptations such as AdaSGHMC. To understand why, we analyze the evolution of AdaSGHMC’s internal mass matrix $\mathbf{M}$. AdaSGHMC dynamically updates $\mathbf{M}$ during warmup to learn the local geometry. Our analysis reveals that this internal adaptation effectively “overwrites” any initial preconditioning. Figure 8 substantiates this claim, displaying the distribution of values in $\mathbf{M}$ after adaptation. The left subplot shows the result when starting from identity preconditioning, while the right subplot starts from our first-moment-based preconditioning strategy. Visually, the two distributions are nearly indistinguishable, showing that marginalized over the layers, the sampler results in the same geometric configuration regardless of the starting point.

We further quantify this phenomenon using pairwise cosine similarity analysis across ensemble chains (see Figure 9), uncovering two key insights. First, the analysis validates our overwriting hypothesis: after adaptation, mass matrices initialized from identity and from the first moment are highly similar, with nearly identical third and fourth boxes in the left and right subplot each. Second, AdaSGHMC’s adaptation actively reduces ensemble diversity. The adapted matrices exhibit extremely high self-similarity (near 1 with almost no variance in the right subplot), meaning mass matrices become almost identical across all chains. In contrast, the static first-moment preconditioner (second box of the left subplot) shows lower self-similarity values. We hypothesize this preserves unique geometric information from each chain’s starting mode. This loss of diversity enforced by scale-adaptation likely explains why our simpler, diversity-preserving preconditioned SGHMC often achieves superior generalization performance.

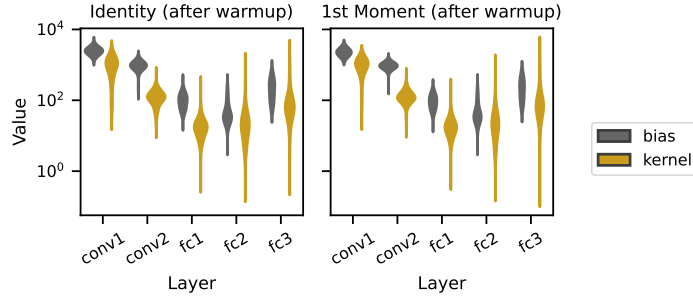


Figure 8: Layer-wise distributions of bias and kernel parameters on the diagonal of the corresponding mass matrices after warmup adaptation throughout the layers. We show the values of $\mathbf{M}$ after identity- (left) and after first moment-based preconditioning (right).

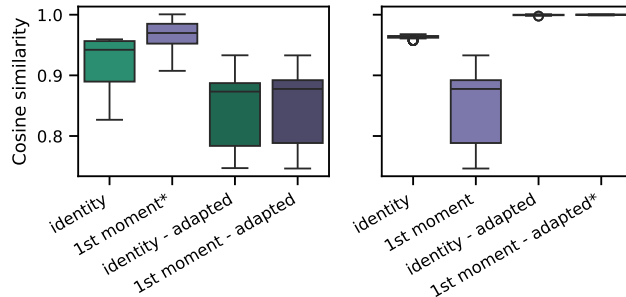


Figure 9: Cosine similarity of $\mathbf{M}$ matrices after pairwise comparison with the first moment-based preconditioning matrix (left) and the adapted first moment-based preconditioning matrix (right). Self-similarities are marked by a star (*).

D.6 Performance Overview of Layer-wise Preconditioning

Since we provide only average performance scores and the improvement over the DE baseline Table 1, we detail our results here, including also the standard error over three runs. Tables 6 and 7 present these details for

predictive performance and uncertainty quantification, respectively. The results further confirm the consistent improvements achieved by our proposed preconditioning strategies, as discussed above.

Table 6: Predictive performance of layer-wise preconditioning across datasets and inference methods. Reported are RMSE ($\downarrow$) for regression tasks and Accuracy ($\uparrow$) for classification tasks. Values denote mean $\pm$ standard error over three independent runs.

Sampler	Preconditioning	RMSE ($\downarrow$)		Accuracy ($\uparrow$)		
		bike	protein	CIFAR-10	F-MNIST	shakespeare
Deep Ensemble	–	0.2445 $\pm$ 0.0089	0.6589 $\pm$ 0.0015	0.9019 $\pm$ 0.0001	0.9157 $\pm$ 0.0015	0.5504 $\pm$ 0.0030
SGHMC	identity	0.2392 $\pm$ 0.0090	0.6533 $\pm$ 0.0049	0.9150 $\pm$ 0.0014	0.9226 $\pm$ 0.0004	0.5536 $\pm$ 0.0022
SGHMC	gradient	0.2406 $\pm$ 0.0089	0.6492 $\pm$ 0.0033	0.9155 $\pm$ 0.0007	0.9268 $\pm$ 0.0002	0.5533 $\pm$ 0.0029
SGHMC	1st moment	0.2405 $\pm$ 0.0087	0.6504 $\pm$ 0.0028	0.9141 $\pm$ 0.0004	0.9256 $\pm$ 0.0011	0.5543 $\pm$ 0.0022
SGHMC	2nd moment	0.2400 $\pm$ 0.0089	0.6470 $\pm$ 0.0035	0.9154 $\pm$ 0.0012	0.9231 $\pm$ 0.0004	0.5535 $\pm$ 0.0016
AdaSGHMC	adaptive	0.2424 $\pm$ 0.0087	0.6560 $\pm$ 0.0023	0.9055 $\pm$ 0.0025	0.9197 $\pm$ 0.0005	0.5523 $\pm$ 0.0027

Table 7: Uncertainty quantification performance of layer-wise preconditioning across datasets and inference methods. Reported are LPPD ($\uparrow$) and perplexity ($\downarrow$). Values denote mean $\pm$ standard error over three independent runs.

Sampler	Preconditioning	LPPD ($\uparrow$)				Perplexity ($\downarrow$)
		bike	protein	CIFAR-10	F-MNIST	shakespeare
Deep Ensemble	–	0.5834 $\pm$ 0.0206	-0.7173 $\pm$ 0.0143	-0.3047 $\pm$ 0.0025	-0.2318 $\pm$ 0.0002	4.3687 $\pm$ 0.0310
SGHMC	identity	0.6190 $\pm$ 0.0163	-0.6434 $\pm$ 0.0086	-0.2641 $\pm$ 0.0025	-0.2216 $\pm$ 0.0002	4.2645 $\pm$ 0.0250
SGHMC	gradient	0.6418 $\pm$ 0.0179	-0.6171 $\pm$ 0.0210	-0.2583 $\pm$ 0.0012	-0.2102 $\pm$ 0.0008	4.2846 $\pm$ 0.0272
SGHMC	1st moment	0.6416 $\pm$ 0.0172	-0.6274 $\pm$ 0.0236	-0.2644 $\pm$ 0.0012	-0.2118 $\pm$ 0.0001	4.2801 $\pm$ 0.0287
SGHMC	2nd moment	0.6397 $\pm$ 0.0179	-0.6002 $\pm$ 0.0210	-0.2615 $\pm$ 0.0020	-0.2214 $\pm$ 0.0002	4.2894 $\pm$ 0.0412
AdaSGHMC	adaptive	0.6182 $\pm$ 0.0192	-0.6908 $\pm$ 0.0173	-0.2942 $\pm$ 0.0011	-0.2197 $\pm$ 0.0016	4.3382 $\pm$ 0.0382

D.7 Performance Overview of Parameter-wise Preconditioning

Tables 8 and 9 detail the results when applying parameter- instead of layer-wise preconditioning. The missing entries highlight the numerical instabilities encountered with this more granular strategy on several datasets. This empirically validates our choice to use the layer-wise approach for the main experiments, as it proved to be more robust and did not suffer from such failures in any of our tested configurations.

Table 8: Predictive performance of parameter-wise preconditioning across datasets. Reported are the RMSE ($\downarrow$) for regression tasks and Accuracy ($\uparrow$) for classification tasks. Values denote mean $\pm$ standard error over three independent runs.

Sampler	Preconditioning	RMSE ($\downarrow$)		Accuracy ($\uparrow$)		
		bike	protein	CIFAR-10	F-MNIST	shakespeare
Deep Ensemble	–	0.2445 $\pm$ 0.0089	0.6589 $\pm$ 0.0015	0.9019 $\pm$ 0.0001	0.9157 $\pm$ 0.0015	0.5504 $\pm$ 0.0030
SGHMC	identity	0.2392 $\pm$ 0.0090	0.6533 $\pm$ 0.0049	0.9150 $\pm$ 0.0014	0.9226 $\pm$ 0.0004	0.5536 $\pm$ 0.0022
SGHMC	gradient	–	–	0.9128 $\pm$ 0.0020	–	0.5535 $\pm$ 0.0020
SGHMC	1st moment	–	–	0.9153 $\pm$ 0.0006	–	0.5524 $\pm$ 0.0017
SGHMC	2nd moment	0.2426 $\pm$ 0.0090	0.6510 $\pm$ 0.0037	0.9150 $\pm$ 0.0009	0.9228 $\pm$ 0.0012	0.5539 $\pm$ 0.0015

Table 9: Uncertainty quantification performance of parameter-wise preconditioning across datasets. Reported are LPPD ($\uparrow$) and perplexity ($\downarrow$). Values denote mean $\pm$ standard error over three independent runs.

Sampler	Preconditioning	LPPD ($\uparrow$)				Perplexity ($\downarrow$)
		bike	protein	CIFAR-10	F-MNIST	shakespeare
Deep Ensemble	–	0.5834 $\pm$ 0.0206	-0.7173 $\pm$ 0.0143	-0.3047 $\pm$ 0.0025	-0.2318 $\pm$ 0.0002	4.3687 $\pm$ 0.0310
SGHMC	identity	0.6190 $\pm$ 0.0163	-0.6434 $\pm$ 0.0086	-0.2641 $\pm$ 0.0025	-0.2216 $\pm$ 0.0002	4.2645 $\pm$ 0.0250
SGHMC	gradient	–	–	-0.2678 $\pm$ 0.0057	–	4.2770 $\pm$ 0.0294
SGHMC	1st moment	–	–	-0.2635 $\pm$ 0.0017	–	4.2871 $\pm$ 0.0238
SGHMC	2nd moment	0.5646 $\pm$ 0.0104	-0.6223 $\pm$ 0.0192	-0.2635 $\pm$ 0.0026	-0.2213 $\pm$ 0.0007	4.2964 $\pm$ 0.0211

E OPTIMIZATION DETAILS

We provide pseudo-code for the AdamW optimizer (Algorithm 1) using notation aligned with the formalization of the preconditioning schemes. Presenting the optimizer in this form makes explicit that the first- and second-moment statistics required by our optimizer-informed preconditioning are readily available as part of the standard optimization state. In particular, the terminal values of the moment accumulators $\boldsymbol{\mu}_t$ and $\boldsymbol{\nu}_t$ can be extracted at no additional computational cost and directly reused to initialize the sampler’s mass matrix.

Algorithm 1 AdamW (Loshchilov and Hutter, 2019)

Require: Learning rate $\alpha > 0$, Momentum parameters $\beta_1, \beta_2 \in [0, 1)$, Weight decay $\lambda \geq 0$, Stability constant $\epsilon > 0$

```

1: Init:  $\boldsymbol{\theta}_0 \leftarrow$  initial parameters,  $\boldsymbol{\mu}_0 \leftarrow \mathbf{0}$  (first moment),  $\boldsymbol{\nu}_0 \leftarrow \mathbf{0}$  (second moment)
2: for  $t = 1, 2, \dots$  do
3:    $\mathbf{g}_t \leftarrow \nabla \ell(\boldsymbol{\theta}_{t-1})$ 
4:    $\boldsymbol{\mu}_t \leftarrow \beta_1 \boldsymbol{\mu}_{t-1} + (1 - \beta_1) \mathbf{g}_t$ 
5:    $\boldsymbol{\nu}_t \leftarrow \beta_2 \boldsymbol{\nu}_{t-1} + (1 - \beta_2) \mathbf{g}_t^2$ 
6:    $\boldsymbol{\mu}'_t \leftarrow \boldsymbol{\mu}_t / (1 - \beta_1^t)$ 
7:    $\boldsymbol{\nu}'_t \leftarrow \boldsymbol{\nu}_t / (1 - \beta_2^t)$ 
8:    $\boldsymbol{\theta}_t \leftarrow \boldsymbol{\theta}_{t-1} - \alpha(\boldsymbol{\mu}'_t + \lambda \boldsymbol{\theta}_{t-1}) / (\sqrt{\boldsymbol{\nu}'_t} + \epsilon)$ 
9: end for
10: return  $\boldsymbol{\theta}_t$ 
```
